

Transformers and LLMs 2

Interview-focused guide for modern LLM training, adaptation, inference, retrieval, tooling, and deployment topics.

RLHF

1. Overview

Reinforcement Learning from Human Feedback (RLHF) aligns a pretrained or instruction-tuned language model with human preferences. It is useful when "correctness" is hard to express with a single supervised label, such as helpfulness, harmlessness, tone, refusal behavior, and response quality. Real systems use RLHF or RLHF-like post-training to make chatbots, coding assistants, tutoring agents, summarizers, and customer-support models more useful and safer.

2. Intuition

First teach the model to answer with examples, then ask humans which of two answers is better, then train the model to prefer answers that humans would choose. Analogy: supervised fine-tuning teaches a student by showing solved answers; RLHF is like giving grades and comments, then making the student optimize for better grades.

3. Prerequisites

Transformers, language-model loss, supervised fine-tuning, policy gradients, reward models, KL divergence, preference datasets, PyTorch/Hugging Face basics.

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
Policy model	The LLM being optimized	Generates candidate responses	Chat model	Why call an LLM a policy?
Preference data	Human-ranked outputs	Captures subjective quality	A preferred over B	Pairwise ranking loss
Reward model	Predicts human preference score	Replaces expensive human scoring during RL	$r_\phi(x, y)$	Reward hacking risk
PPO	Stable RL optimizer often used in RLHF	Updates policy without drifting too far	KL penalty	Why KL to reference model?
Reference model	Frozen copy of SFT model	Prevents collapse and weird outputs	π_{ref}	Alignment vs capability tradeoff

5. Algorithm / Working Process

1. Start with a pretrained LLM.
2. Supervised fine-tune it on high-quality instruction-response data.
3. Collect prompts and multiple model responses.
4. Ask humans or labelers to rank responses.
5. Train a reward model to predict the preferred response.
6. Optimize the policy model with RL, commonly PPO, using reward plus KL penalty.
7. Evaluate with human preference, safety tests, factuality checks, and regression tests.

Input: prompt. Processing: policy generates response, reward model scores it, PPO updates model. Output: aligned chat model.

6. Mathematical Foundation

Preference reward model commonly uses Bradley-Terry loss:

$$P(y_w \succ y_l | x) = \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))$$

$$L_{RM} = -\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))$$

RLHF objective:

$$\max_{\theta} E_{y \sim \pi_{\theta}(\cdot | x)} [r_{\phi}(x, y)] - \beta KL(\pi_{\theta}(\cdot | x) || \pi_{ref}(\cdot | x))$$

The KL term keeps the optimized model close to the original instruction-tuned model.

7. Practical Implementation

```
# Minimal preference reward-model loss in PyTorch
import torch
import torch.nn.functional as F

def reward_model_loss(chosen_rewards, rejected_rewards):
    # chosen_rewards, rejected_rewards: shape [batch]
    return -F.logsigmoid(chosen_rewards - rejected_rewards).mean()

chosen = torch.tensor([3.0, 1.2, 2.1])
rejected = torch.tensor([1.0, 1.5, 0.4])
loss = reward_model_loss(chosen, rejected)
print(loss.item())
```

8. Code Explanation

`chosen_rewards - rejected_rewards` should be positive when the reward model agrees with human labels. `logsigmoid` creates the pairwise ranking loss. The mean gives one scalar training objective.

9. Training / Evaluation

Prepare prompts with chosen/rejected responses. Split by prompt, not by individual response, to avoid leakage. Evaluate reward-model accuracy on held-out pairs, then evaluate final LLM using preference win rate,

safety tests, factuality, refusal precision/recall, and task benchmarks. Watch for over-optimization: reward goes up while real response quality drops.

10. Complexity and Cost

RLHF is expensive: it needs human labels, reward-model training, and repeated LLM generation during RL. Memory cost is high because PPO may keep policy, reference, reward, and value models. Inference cost after training is usually the same as the base model.

11. Common Use Cases

Chat assistants, coding assistants, content moderation, summarization, tutoring, medical-style triage assistants with strict safety boundaries, enterprise support bots.

12. Common Mistakes

Using noisy preference labels, optimizing reward too aggressively, ignoring KL, evaluating only with the reward model, mixing train/test prompts, missing safety evals, treating RLHF as a factuality solution, failing to check distribution shift.

13. Edge Cases / Limitations

RLHF can encode labeler bias, encourage verbose answers, produce reward hacking, reduce diversity, and fail on rare or technical prompts. It improves preference alignment, not guaranteed truth.

14. Variations

RLAIF uses AI feedback instead of human feedback. Constitutional AI uses rule-based critique and revision. DPO removes explicit RL. IPO/KTO/ORPO are direct preference optimization variants. PPO-style RLHF remains important for research and large post-training stacks.

15. Related Topics

SFT teaches imitation; RLHF optimizes preference. DPO is a simpler alternative. Reward modeling connects to ranking. Safety training connects to refusal behavior and red teaming.

16. Interview Questions

1. What problem does RLHF solve? It aligns outputs with human preferences when labels are subjective.
2. Why train a reward model? Human scoring every RL sample is too expensive.
3. What is the KL penalty for? It prevents the policy from drifting too far from a stable reference.
4. Why use pairwise preferences? Ranking is easier and more reliable than absolute scoring.
5. What is reward hacking? The model exploits the learned reward instead of genuinely improving.
6. Is RLHF enough for factuality? No, it improves preferred behavior but does not guarantee truth.
7. What comes before RLHF? Pretraining and usually supervised fine-tuning.
8. What is PPO's role? It updates the policy with controlled steps.
9. How do you evaluate RLHF? Human win rate, safety tests, benchmark regressions, and reward-model diagnostics.
10. What is a reference model? A frozen baseline used for KL regularization.

17. Practice Tasks

Implement pairwise reward loss. Build a small preference dataset for summaries. Compare SFT output vs reward-ranked output. Debug a reward model that always prefers longer answers. Extend with DPO.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Summary Preference Trainer	Trains reward model for summaries	PyTorch, Transformers	TL;DR preference data	Shows alignment basics
Helpful Chat Ranker	Ranks chatbot answers	HF, PEFT	Anthropic HH subset	Practical eval pipeline
Reward Hacking Demo	Shows over-optimization failure	PyTorch	Synthetic prompts	Research maturity

19. Quick Revision

Key idea: optimize LLM behavior using human preferences. Main formula: reward minus KL penalty. Use when response quality is subjective. Metrics: win rate, reward accuracy, safety pass rate. Trap: trusting reward score alone. Interview one-liner: RLHF turns human preferences into a reward signal for post-training.

20. Final Cheat Sheet

Definition: preference-based alignment. Input/output: prompt to preferred response. Main steps: SFT, preference data, reward model, PPO. Hyperparameters: KL beta, learning rate, rollout length. Pros: better helpfulness. Cons: costly, reward hacking. Best use: chat alignment.

DPO

1. Overview

Direct Preference Optimization (DPO) trains an LLM directly from chosen/rejected preference pairs without training a separate reward model or running online RL. It is popular because it is simpler, more stable, and cheaper than PPO-based RLHF for many alignment tasks.

2. Intuition

If humans prefer answer A over B for the same prompt, increase the model's probability of A and decrease the probability of B, while staying close to a reference model. DPO bakes the reward-model math into a supervised-looking loss.

3. Prerequisites

Cross-entropy training, log probabilities, preference pairs, KL regularization, SFT, token-level likelihoods.

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
Chosen/rejected	Preferred and non-preferred response	Main training signal	Good answer vs bad answer	Pairwise learning
Reference model	Usually frozen SFT model	Controls drift	π_{ref}	Why not train only chosen?
Log-prob ratio	Preference strength under model vs reference	Core DPO signal	$\log \pi(y_w) - \log \pi(y_l)$	Derive loss intuition
Beta	Controls deviation from reference	Higher beta sharpens preference pressure	$\beta = 0.1$	Tuning stability

5. Algorithm / Working Process

1. Start with an SFT model and freeze a copy as reference.
2. For each prompt, store chosen and rejected answers.
3. Compute policy log probability of both answers.
4. Compute reference log probability of both answers.
5. Optimize DPO loss to prefer chosen over rejected relative to reference.
6. Evaluate with preference win rate and task benchmarks.

6. Mathematical Foundation

DPO loss:

$$L_{DPO} = -E[\log \sigma(\beta((\log \pi_{\theta}(y_w|x) - \log \pi_{\theta}(y_l|x)) - (\log \pi_{ref}(y_w|x) - \log \pi_{ref}(y_l|x))))]$$

It increases the relative probability of the preferred answer more than the reference model does.

7. Practical Implementation

```
import torch
import torch.nn.functional as F

def dpo_loss(policy_chosen, policy_rejected, ref_chosen, ref_rejected, beta=0.1):
    policy_logratios = policy_chosen - policy_rejected
    ref_logratios = ref_chosen - ref_rejected
    logits = beta * (policy_logratios - ref_logratios)
    return -F.logsigmoid(logits).mean()

loss = dpo_loss(
    torch.tensor([-12.0, -8.0]),
    torch.tensor([-15.0, -7.5]),
    torch.tensor([-13.0, -8.2]),
    torch.tensor([-14.0, -7.7]),
```

```
)  
print(loss.item())
```

8. Code Explanation

The model compares chosen and rejected sequence log probabilities. The reference comparison is subtracted, so the policy is rewarded only for improving preference relative to the baseline.

9. Training / Evaluation

Use clean preference pairs. Split by prompt. Track DPO loss, chosen-vs-rejected accuracy, validation win rate, length bias, and benchmark regressions. Tune beta and learning rate carefully.

10. Complexity and Cost

DPO is cheaper than PPO RLHF because it is offline and avoids rollout generation during training. It still needs two forward passes through policy and reference, or cached reference logprobs.

11. Common Use Cases

Chat alignment, style tuning, summarization preference tuning, code assistant response ranking, domain assistant refinement.

12. Common Mistakes

Bad preference data, forgetting the reference model, using token logprobs incorrectly, comparing prompts with different responses without masking, overfitting small preference sets, ignoring length bias.

13. Edge Cases / Limitations

DPO depends heavily on preference quality. It may be weaker than RL for tasks needing exploration or external rewards. It can still reduce diversity or overfit style preferences.

14. Variations

IPO changes the preference objective. KTO uses desirable/undesirable examples without strict pairs. ORPO combines supervised and preference losses. SimPO removes the explicit reference in some formulations.

15. Related Topics

DPO vs RLHF: DPO is offline and simpler; RLHF uses reward model plus RL. DPO vs SFT: SFT imitates chosen answers; DPO learns relative preferences.

16. Interview Questions

1. What is DPO? Direct training on preference pairs.
2. Why is it simpler than RLHF? No reward model or PPO loop.
3. What data does DPO need? Prompt, chosen response, rejected response.
4. Why use a reference model? To regularize behavior and preserve capabilities.

- 5. What does beta do? Controls preference strength vs reference closeness.
- 6. How is DPO evaluated? Win rate and downstream benchmarks.
- 7. Does DPO need online sampling? No, it is usually offline.
- 8. Can DPO replace SFT? Usually no; it commonly follows SFT.
- 9. What is the risk of noisy pairs? The model learns wrong preferences.
- 10. Why compute sequence logprob? Preference applies to full response likelihood.

17. Practice Tasks

Implement DPO loss. Cache reference logprobs. Fine-tune a small model on synthetic preference pairs. Analyze length bias. Compare SFT-only vs SFT+DPO.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
DPO Chat Tuner	Fine-tunes small chat model	TRL, PEFT	HH-RLHF subset	Modern alignment skill
Style Preference Model	Tunes concise answers	HF	Custom pairs	Product relevance
DPO Loss Lab	Visualizes beta effects	PyTorch, Streamlit	Synthetic	Math clarity

19. Quick Revision

Key idea: increase probability of preferred response relative to rejected and reference. Formula: DPO sigmoid loss on log-ratio difference. Use when you have preference pairs. Trap: bad logprob masking.

20. Final Cheat Sheet

Definition: offline preference optimization. Input/output: preference pairs to aligned model. Steps: compute policy/ref logprobs, optimize DPO loss. Hyperparameters: beta, LR, batch size. Pros: simpler than RLHF. Cons: data-sensitive.

LoRA

1. Overview

Low-Rank Adaptation (LoRA) fine-tunes large neural networks by freezing original weights and training small low-rank update matrices. It is widely used to adapt LLMs cheaply for domains, styles, tasks, and instruction tuning.

2. Intuition

Instead of changing a huge matrix directly, learn a small correction that can be written as the product of two skinny matrices. It is like adding a small steering attachment to a large machine instead of rebuilding the

machine.

3. Prerequisites

Matrix multiplication, rank, linear layers, Transformer attention, fine-tuning, PyTorch modules.

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
Frozen base	Original model weights are not trained	Saves memory	Llama weights frozen	Why memory drops
Low-rank update	$\Delta W = BA$	Few trainable params	rank 8 adapter	Rank tradeoff
Target modules	Layers receiving LoRA	Controls quality/cost	q_proj, v_proj	Where to apply LoRA
Merge	Add LoRA delta into base weights	Faster inference	merged checkpoint	Deployment

5. Algorithm / Working Process

For a linear layer $y = Wx$, freeze W . Add trainable matrices $A \in R^{r \times d}$ and $B \in R^{k \times r}$. During forward pass, compute $y = Wx + \alpha/r \cdot BAx$. Train only A and B .

6. Mathematical Foundation

$$W' = W + \Delta W, \quad \Delta W = \frac{\alpha}{r} BA$$

If W has shape $k \times d$, LoRA trains $r(d + k)$ parameters instead of kd . With $r \ll \min(d, k)$, parameter savings are large.

7. Practical Implementation

```
from peft import LoraConfig, get_peft_model
from transformers import AutoModelForCausalLM

model = AutoModelForCausalLM.from_pretrained("gpt2")

config = LoraConfig(
    r=8,
    lora_alpha=16,
    target_modules=["c_attn"],
    lora_dropout=0.05,
    task_type="CAUSAL_LM",
)
```

```
model = get_peft_model(model, config)
model.print_trainable_parameters()
```

8. Code Explanation

`r` is adapter rank. `lora_alpha` scales the update. `target_modules` chooses which linear layers receive adapters. `get_peft_model` freezes base weights and inserts trainable LoRA modules.

9. Training / Evaluation

Prepare instruction or domain data. Use normal language-model loss. Evaluate task accuracy, perplexity, human preference, latency, and whether the adapter overfits. Tune rank, alpha, dropout, target modules, and learning rate.

10. Complexity and Cost

Training memory is much lower than full fine-tuning because gradients and optimizer states are stored only for adapters. Inference has slight overhead unless adapters are merged.

11. Common Use Cases

Domain adaptation, instruction tuning, style tuning, personalization, multilingual adaptation, code assistant tuning.

12. Common Mistakes

Targeting wrong module names, using too high rank for small data, forgetting to save adapter config, evaluating only training loss, mixing incompatible base model and adapter, expecting LoRA to add missing knowledge perfectly.

13. Edge Cases / Limitations

LoRA may underperform full fine-tuning for large distribution shifts. It still needs quality data. Many adapters loaded at once can complicate serving.

14. Variations

AdaLoRA changes rank adaptively. DoRA separates magnitude and direction. QLoRA combines LoRA with quantized base weights. IA3 trains multiplicative vectors instead of low-rank matrices.

15. Related Topics

LoRA vs full fine-tuning: cheaper but less flexible. LoRA vs QLoRA: QLoRA quantizes base model too. LoRA is a PEFT method.

16. Interview Questions

1. What does LoRA train? Low-rank adapter matrices.
2. Why freeze base weights? To reduce memory and preserve base capability.

3. What is rank r ? Size of the adapter bottleneck.
4. What is alpha? Scaling factor for LoRA update.
5. Can LoRA be merged? Yes, add delta to base weights.
6. Where is LoRA applied in LLMs? Attention and sometimes MLP projections.
7. Does LoRA reduce inference memory? Adapter is small; base still needed.
8. Why is it parameter efficient? It trains $r(d + k)$ not dk parameters.
9. What happens if r is too small? Underfitting.
10. What happens if r is too large? More memory and overfitting risk.

17. Practice Tasks

Fine-tune GPT-2 with LoRA. Compare ranks 4, 8, 16. Merge adapter and test generation. Inspect trainable parameter count. Debug wrong `target_modules`.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Domain LoRA Bot	Tunes model for one domain	PEFT, Transformers	Company FAQ	Practical fine-tuning
Rank Ablation Study	Compares LoRA ranks	PyTorch	Alpaca subset	Research mindset
Multi-Adapter Demo	Switches between styles	PEFT	Custom style data	Deployment skill

19. Quick Revision

Key idea: train low-rank deltas while base is frozen. Formula: $W' = W + \alpha BA/r$. Use for cheap fine-tuning. Trap: wrong target modules.

20. Final Cheat Sheet

Definition: parameter-efficient fine-tuning via low-rank updates. Input/output: base model plus task data to adapter. Hyperparameters: r , alpha, dropout, target layers. Pros: cheap. Cons: limited capacity.

QLoRA

1. Overview

Quantized LoRA (QLoRA) fine-tunes a quantized base model using LoRA adapters. The base model is stored in low precision, commonly 4-bit, while adapter weights train in higher precision. This enables fine-tuning large LLMs on limited GPU memory.

2. Intuition

Keep the heavy model compressed and frozen, then train a small adapter on top. It is like reading a compressed reference book and writing small sticky-note corrections.

3. Prerequisites

LoRA, quantization, mixed precision, GPU memory, optimizer states, Transformers, PEFT.

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
4-bit base	Frozen model stored compactly	Huge memory saving	NF4 weights	Why QLoRA fits on one GPU
LoRA adapters	Trainable small matrices	Adapt task behavior	rank 16	Why not train quantized weights
Double quantization	Quantizes quantization constants	Saves extra memory	bitsandbytes	Memory details
Paged optimizer	Handles memory spikes	Prevents OOM	paged AdamW	Practical training

5. Algorithm / Working Process

Load base model in 4-bit. Insert LoRA adapters. During forward pass, dequantize needed weights for computation, apply LoRA update, compute loss, and update only adapter parameters. Save adapter, not full model.

6. Mathematical Foundation

Quantized base:

$$W_q = Q(W), \quad W \approx \text{dequantize}(W_q)$$

LoRA update:

$$y = \text{dequantize}(W_q)x + \frac{\alpha}{r}BAx$$

NF4 is designed for normally distributed neural weights and gives better 4-bit representation than uniform quantization.

7. Practical Implementation

```
from transformers import AutoModelForCausalLM, BitsAndBytesConfig
from peft import LoraConfig, get_peft_model
import torch

bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
```

```
        bnb_4bit_compute_dtype=torch.bfloat16,
    )

    model = AutoModelForCausalLM.from_pretrained(
        "gpt2",
        quantization_config=bnb_config,
        device_map="auto",
    )

    lora = LoraConfig(r=8, lora_alpha=16, target_modules=["c_attn"],
        task_type="CAUSAL_LM")
    model = get_peft_model(model, lora)
    model.print_trainable_parameters()
```

8. Code Explanation

`load_in_4bit` compresses the frozen base. `nf4` is the 4-bit quantization type. LoRA adds trainable adapters. Only adapter parameters receive gradients.

9. Training / Evaluation

Use instruction data, pack sequences carefully, and monitor memory. Evaluate generation quality, task metrics, adapter overfitting, and latency. Compare with LoRA on higher precision if resources allow.

10. Complexity and Cost

QLoRA greatly reduces memory. Compute may be slower than pure fp16 because of quantization overhead. It enables large-model fine-tuning on consumer GPUs.

11. Common Use Cases

Fine-tuning 7B/13B/70B models with limited GPUs, domain-specific assistants, academic experiments, instruction tuning.

12. Common Mistakes

Trying to update quantized base weights, using unsupported GPU kernels, bad compute dtype, forgetting gradient checkpointing for long context, expecting 4-bit training to equal full fine-tuning always.

13. Edge Cases / Limitations

Quantization can hurt fragile tasks. Some layers may need higher precision. Hardware/kernel support matters. Very small datasets still overfit.

14. Variations

8-bit LoRA is less compressed but often stable. LoftQ initializes LoRA for quantized models. AWQ/GPTQ are more inference-oriented quantization methods.

15. Related Topics

QLoRA = quantization + LoRA + PEFT. It differs from post-training quantization because adapters are trained.

16. Interview Questions

1. What is QLoRA? LoRA fine-tuning on a quantized frozen base.
2. Why use 4-bit? To reduce memory.
3. Are base weights trained? No.
4. What is NF4? A 4-bit format suited to neural weights.
5. Why use bf16 compute? Stability during operations.
6. How does QLoRA differ from LoRA? Base model is quantized.
7. What is double quantization? Quantizing scale constants too.
8. Main risk? Quality loss or kernel incompatibility.
9. What is saved? Adapter weights and config.
10. Best use case? Large-model adaptation on limited hardware.

17. Practice Tasks

Load a small model in 4-bit. Add LoRA. Measure memory. Fine-tune on tiny instruction data. Compare generated outputs with base model.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Single-GPU Fine-Tuner	Tunes a chat model cheaply	PEFT, bitsandbytes	Alpaca subset	Practical LLM training
Quantization Study	Compares 4-bit vs 8-bit	HF	WikiText	Systems awareness
Domain Adapter	Legal/medical-style assistant	QLoRA	Public QA data	Applied AI project

19. Quick Revision

Key idea: quantize base, train adapters. Formula: dequantized base output plus LoRA delta. Use when GPU memory is limited. Trap: unsupported quantization stack.

20. Final Cheat Sheet

Definition: memory-efficient LoRA with quantized base. Input/output: quantized model plus data to adapter. Hyperparameters: rank, alpha, quant type, compute dtype. Pros: cheap. Cons: quantization overhead.

PEFT

1. Overview

Parameter-Efficient Fine-Tuning (PEFT) adapts large models by training only a small number of parameters. It is useful when full fine-tuning is too expensive or when many task-specific adapters must share one base model.

2. Intuition

Instead of rewriting the whole model, attach small trainable parts or tune a small subset of parameters. The base model remains a reusable foundation.

3. Prerequisites

Fine-tuning, Transformers, gradients, adapters, prompts, low-rank matrices.

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
Adapter	Small trainable module	Cheap task adaptation	LoRA adapter	Serving many tasks
Frozen backbone	Shared base model	Saves storage and compute	Llama base	Avoid catastrophic forgetting
Prompt tuning	Train soft prompt embeddings	Very small parameter count	20 virtual tokens	When it works
Prefix tuning	Train key/value prefixes	Influences attention	Prefix vectors	Difference from prompt tuning

5. Algorithm / Working Process

Choose PEFT method, freeze base model, add trainable parameters, train on task data, save small adapter, load adapter with base model at inference.

6. Mathematical Foundation

General PEFT:

$$f_{\theta, \phi}(x), \quad \theta \text{ frozen}, \quad \phi \text{ trainable}, \quad |\phi| \ll |\theta|$$

Optimize:

$$\min_{\phi} L(f_{\theta, \phi}(x), y)$$

7. Practical Implementation

```
from peft import LoraConfig, get_peft_model
from transformers import AutoModelForSequenceClassification

base = AutoModelForSequenceClassification.from_pretrained("distilbert-base-uncased")
```

```
config = LoraConfig(r=4, lora_alpha=8, target_modules=["q_lin", "v_lin"],
task_type="SEQ_CLS")
model = get_peft_model(base, config)
model.print_trainable_parameters()
```

8. Code Explanation

The base classifier is loaded normally. PEFT inserts LoRA modules into attention projections. Training updates only the new parameters.

9. Training / Evaluation

Use the same task metrics as normal fine-tuning: accuracy/F1 for classification, ROUGE/BLEU for generation, exact match for QA, win rate for chat. Validate adapter quality and check for base-model compatibility.

10. Complexity and Cost

PEFT reduces optimizer-state memory and checkpoint size. Full base model is still needed for inference. Adapter switching is cheap compared with loading full model copies.

11. Common Use Cases

Multi-tenant LLM serving, task-specific models, domain adaptation, personalization, low-resource research, edge experiments.

12. Common Mistakes

Assuming PEFT always matches full fine-tuning, training too few parameters for a large shift, poor adapter management, incompatible model architecture names, bad validation splits.

13. Edge Cases / Limitations

Large domain shifts or new skills may require full fine-tuning or continued pretraining. Soft prompts can be brittle. Adapter composition can conflict.

14. Variations

LoRA, QLoRA, adapters, prefix tuning, prompt tuning, IA3, BitFit. LoRA/QLoRA are most placement-relevant.

15. Related Topics

PEFT vs full fine-tuning: cheaper but less expressive. PEFT vs RAG: PEFT changes behavior; RAG injects knowledge at inference.

16. Interview Questions

1. What is PEFT? Fine-tuning a small parameter subset.
2. Why use it? Memory and storage savings.
3. Name PEFT methods. LoRA, adapters, prompt tuning, prefix tuning.
4. Does PEFT need base model at inference? Yes.

5. Is LoRA PEFT? Yes.
6. What is adapter merging? Folding adapter deltas into base weights.
7. PEFT or RAG for new facts? Usually RAG.
8. PEFT or full fine-tuning for new style? PEFT often works.
9. Main limitation? Limited adaptation capacity.
10. How evaluate? Same metrics as target task plus regression tests.

17. Practice Tasks

Train LoRA for classification. Compare full fine-tuning vs PEFT memory. Save/load an adapter. Try prompt tuning on a small model. Measure checkpoint size.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Adapter Zoo	Multiple adapters on one base	PEFT	GLUE tasks	Efficient deployment
PEFT Benchmark	Compares methods	HF	SST-2, AG News	Interview depth
Personal Writing Adapter	Tunes style	LoRA	Own writing samples	Portfolio demo

19. Quick Revision

Key idea: freeze base, train small additions. Formula: optimize ϕ while θ frozen. Use when resources are limited. Trap: expecting it to learn large new knowledge reliably.

20. Final Cheat Sheet

Definition: small-parameter adaptation. Input/output: base model plus task data to adapter. Hyperparameters: method, rank/prefix length, LR. Pros: cheap. Cons: lower capacity.

Quantization

1. Overview

Quantization represents model weights and/or activations with fewer bits, such as int8 or int4 instead of fp16/fp32. It reduces memory, bandwidth, and sometimes latency, making LLM inference cheaper and easier to deploy.

2. Intuition

Instead of storing every weight with high precision, round values into a smaller set of levels. Like storing approximate temperatures instead of many decimal places: usually enough, but too much rounding loses information.

3. Prerequisites

Number formats, linear layers, calibration, inference hardware, matrix multiplication, error analysis.

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
Weight quantization	Compress weights	Reduces model size	int4 weights	Memory savings
Activation quantization	Compress runtime activations	Speeds compute on hardware	int8 activations	Harder than weights
Scale/zero point	Map floats to integers	Controls reconstruction	affine quant	Formula
Calibration	Estimate ranges	Avoid clipping	sample dataset	PTQ quality

5. Algorithm / Working Process

For post-training quantization, collect representative data, estimate value ranges, map floating values to integers, store scale metadata, and run inference with quantized kernels or dequantization.

6. Mathematical Foundation

Uniform affine quantization:

$$q = \text{round}(x/s + z)$$

$$\hat{x} = s(q - z)$$

where s is scale and z is zero point. Quantization error is $e = x - \hat{x}$.

7. Practical Implementation

```
import numpy as np

def quantize_int8(x):
    qmin, qmax = -128, 127
    scale = np.max(np.abs(x)) / qmax
    q = np.clip(np.round(x / scale), qmin, qmax).astype(np.int8)
    return q, scale

def dequantize_int8(q, scale):
    return q.astype(np.float32) * scale

w = np.array([0.1, -0.7, 1.3, 2.0], dtype=np.float32)
q, s = quantize_int8(w)
print(q, dequantize_int8(q, s))
```

8. Code Explanation

The maximum absolute value sets the scale. Values are divided by scale, rounded to int8, clipped, and later reconstructed by multiplying by scale.

9. Training / Evaluation

Evaluate perplexity, task accuracy, latency, memory, and throughput before and after quantization. Use representative calibration data. For sensitive models, use quantization-aware training.

10. Complexity and Cost

Memory roughly scales with bit-width: int4 uses about one quarter of fp16 weight memory. Speedup depends on hardware kernels. Quantization can reduce bandwidth bottlenecks.

11. Common Use Cases

LLM serving, mobile/edge deployment, GPU memory reduction, CPU inference, embedding model deployment.

12. Common Mistakes

Calibrating on unrepresentative data, quantizing sensitive layers blindly, ignoring activation outliers, measuring size but not quality, assuming lower bits always mean faster inference.

13. Edge Cases / Limitations

Outlier-heavy activations can break low-bit quantization. Some tasks are more precision-sensitive. Kernel support may dominate real speed.

14. Variations

PTQ quantizes after training. QAT simulates quantization during training. GPTQ, AWQ, SmoothQuant, NF4, int8, int4, weight-only quantization.

15. Related Topics

Quantization vs pruning: fewer bits vs fewer weights. Quantization vs distillation: compression by precision vs smaller student model. QLoRA uses quantized base weights.

16. Interview Questions

1. What is quantization? Lower-precision numeric representation.
2. Why use it? Lower memory and sometimes faster inference.
3. What are scale and zero point? Parameters mapping floats to integers.
4. PTQ vs QAT? After training vs training with quantization simulation.
5. Why calibrate? To estimate ranges.
6. Why are activations hard? They vary by input and may have outliers.
7. Does int4 always speed up? No, depends on kernels.
8. What metric matters? Quality plus latency/memory.

9. What is weight-only quantization? Only weights are low-bit.
10. How does QLoRA use quantization? It freezes a 4-bit base and trains adapters.

17. Practice Tasks

Implement int8 quantization. Quantize a small neural net. Measure accuracy drop. Compare per-tensor vs per-channel scales. Test calibration data choices.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Quantization Lab	Compares int8/int4	PyTorch	MNIST/WikiText	Systems knowledge
CPU LLM Serving	Runs quantized small LLM	llama.cpp/Ollama	Local prompts	Deployment skill
Calibration Study	Tests calibration sets	PyTorch	ImageNet subset	Practical rigor

19. Quick Revision

Key idea: store/compute with fewer bits. Formula: $q = \text{round}(x/s + z)$. Use for cheaper inference. Trap: ignoring quality regression.

20. Final Cheat Sheet

Definition: low-bit model representation. Input/output: float weights to quantized weights. Hyperparameters: bit-width, scale granularity, calibration set. Pros: memory savings. Cons: accuracy loss risk.

KV Cache

1. Overview

The key-value (KV) cache stores previously computed attention keys and values during autoregressive generation. It avoids recomputing attention projections for earlier tokens, making LLM decoding much faster.

2. Intuition

When generating token 100, tokens 1-99 have already been processed. KV cache saves their attention information so the model only computes new work for token 100.

3. Prerequisites

Self-attention, autoregressive decoding, Transformer layers, memory layout, batch inference.

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
Key cache	Stored K vectors per layer	Reused in attention	shape batch, heads, seq, dim	Memory formula
Value cache	Stored V vectors per layer	Produces context	same shape	Why both K and V
Prefill	Process prompt tokens	Builds initial cache	long prompt	Time-to-first-token
Decode	Generate one token at a time	Uses cache	next token loop	Latency

5. Algorithm / Working Process

Prefill computes K/V for all prompt tokens and stores them. During each decode step, compute Q/K/V for the new token, append new K/V to cache, attend from new Q to all cached K/V, sample next token.

6. Mathematical Foundation

Attention:

$$Attention(Q, K, V) = softmax(QK^T / \sqrt{d_k})V$$

Without cache, K and V are recomputed for all previous tokens each step. With cache, only new K_t, V_t are computed and concatenated with cached tensors.

KV memory roughly:

$$2 \times L \times H \times T \times d_h \times bytes$$

where 2 is K and V, L layers, H heads, T sequence length.

7. Practical Implementation

```
from transformers import AutoModelForCausalLM, AutoTokenizer

tok = AutoTokenizer.from_pretrained("gpt2")
model = AutoModelForCausalLM.from_pretrained("gpt2")

inputs = tok("KV cache speeds up decoding because", return_tensors="pt")
out = model.generate(
    **inputs,
    max_new_tokens=20,
    use_cache=True,
)
print(tok.decode(out[0], skip_special_tokens=True))
```

8. Code Explanation

`use_cache=True` tells the model to return and reuse `past_key_values` internally during generation. Hugging Face handles cache updates in the decode loop.

9. Training / Evaluation

KV cache is mainly inference-time. Evaluate tokens/sec, time-to-first-token, decode latency, memory usage, and max batch size. Training usually processes full sequences in parallel and does not use the same decode cache.

10. Complexity and Cost

KV cache reduces repeated computation but increases memory linearly with sequence length, layers, heads, and batch size. Long-context serving can become memory-bound.

11. Common Use Cases

Chatbots, streaming generation, code completion, batch LLM serving, long conversations.

12. Common Mistakes

Forgetting cache memory cost, mixing caches across users, wrong attention masks with padding, not resetting cache between conversations, assuming cache helps prefill as much as decoding.

13. Edge Cases / Limitations

Very long contexts can exhaust GPU memory. Beam search multiplies cache usage. Some architectures use grouped-query attention to reduce cache size.

14. Variations

PagedAttention stores KV blocks efficiently. Multi-query attention and grouped-query attention reduce KV heads. Sliding-window attention evicts old cache tokens.

15. Related Topics

KV cache connects to attention, long-context methods, speculative decoding, and serving engines like vLLM.

16. Interview Questions

1. What is KV cache? Stored keys and values for past tokens.
2. Why useful? Avoids recomputing old K/V during decoding.
3. Does it reduce memory? No, it increases memory.
4. Does it help prefill? Mostly helps decode.
5. Memory scales with what? Layers, heads, sequence length, batch, dtype.
6. What are `past_key_values`? Cached tensors in HF models.
7. Why cache both K and V? Attention needs both similarity and weighted values.
8. How does beam search affect cache? Multiplies cache by beams.
9. What is PagedAttention? Block-based KV memory management.
10. What can reduce KV memory? MQA/GQA, quantized KV, sliding window.

17. Practice Tasks

Generate with and without cache. Measure tokens/sec. Inspect `past_key_values` shapes. Estimate memory for a model. Test long prompt behavior.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Decode Profiler	Measures cache speedup	Transformers	Prompts	Serving insight
KV Memory Calculator	Estimates GPU usage	Python, Streamlit	Model configs	Practical tool
Chat Server Demo	Streams with cache	FastAPI, HF	Local prompts	Production skill

19. Quick Revision

Key idea: reuse past K/V. Formula: attention uses cached K , V . Use for autoregressive decoding. Trap: cache memory explosion.

20. Final Cheat Sheet

Definition: stored attention K/V tensors. Input/output: previous tokens to reusable cache. Hyperparameters: max length, dtype, batch. Pros: faster decoding. Cons: high memory.

FlashAttention Idea

1. Overview

FlashAttention is an exact attention algorithm that reduces memory reads/writes by tiling attention computation on GPU SRAM. It computes the same attention result as standard attention but avoids materializing the full $N \times N$ attention matrix in high-bandwidth memory.

2. Intuition

Standard attention writes a huge score matrix to memory, then reads it back. FlashAttention processes blocks at a time, keeping intermediate values in fast memory, like doing arithmetic on a small desk instead of constantly walking to a storage room.

3. Prerequisites

Self-attention, softmax, GPU memory hierarchy, matrix multiplication, numerical stability.

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
Tiling	Block-wise Q,K,V processing	Improves memory locality	128-token block	Why faster
Online softmax	Stable softmax across blocks	Avoids full score matrix	running max/sum	Exactness
IO awareness	Optimize memory traffic	Attention is memory-bound	HBM vs SRAM	Key paper idea
Exact attention	Same math result	Not approximation	no sparse mask needed	Flash vs sparse

5. Algorithm / Working Process

Split Q, K, V into blocks. For each Q block, iterate over K/V blocks, compute partial scores, update running softmax statistics, and accumulate output. Never store the full attention matrix.

6. Mathematical Foundation

Standard attention:

$$O = \text{softmax}(QK^T / \sqrt{d})V$$

FlashAttention computes this exactly using block-wise online softmax. For numerical stability:

$$m_{\text{new}} = \max(m_{\text{old}}, m_{\text{block}})$$

Running sums are rescaled when the max changes.

7. Practical Implementation

```
import torch
import torch.nn.functional as F

q = torch.randn(2, 8, 128, 64, device="cuda", dtype=torch.float16)
k = torch.randn(2, 8, 128, 64, device="cuda", dtype=torch.float16)
v = torch.randn(2, 8, 128, 64, device="cuda", dtype=torch.float16)

# PyTorch uses optimized scaled dot-product attention when available.
out = F.scaled_dot_product_attention(q, k, v, is_causal=True)
print(out.shape)
```

8. Code Explanation

`scaled_dot_product_attention` dispatches to optimized kernels when hardware and shapes support them. The user writes normal attention code while the backend chooses efficient implementation.

9. Training / Evaluation

FlashAttention affects speed and memory, not model objective. Evaluate max sequence length, training throughput, memory usage, numerical consistency, and supported masks/dtypes.

10. Complexity and Cost

Compute complexity remains $O(N^2d)$ for full attention, but memory IO is much lower. This enables longer sequences and larger batches.

11. Common Use Cases

LLM training, long-context fine-tuning, efficient inference prefill, vision transformers with large token counts.

12. Common Mistakes

Calling it approximate, expecting asymptotic compute reduction, using unsupported masks, ignoring dtype/hardware requirements, benchmarking without synchronization.

13. Edge Cases / Limitations

Still quadratic in sequence length. Kernel support varies. Some custom attention masks may not be supported by fused kernels.

14. Variations

FlashAttention-2 improves parallelism. FlashAttention-3 targets newer GPUs. Memory-efficient attention in xFormers follows similar goals.

15. Related Topics

FlashAttention vs sparse attention: exact IO optimization vs approximate or structured reduction. Connects to long-context training and GPU kernels.

16. Interview Questions

1. What is FlashAttention? IO-aware exact attention.
2. Does it approximate attention? No.
3. What problem does it solve? Memory bandwidth and activation memory.
4. Does it change $O(N^2)$ compute? No for dense attention.
5. Why use online softmax? Stable block-wise computation.
6. Where is full attention matrix stored? It is not materialized.
7. Main benefit? Longer sequences and faster training.
8. Hardware dependency? Needs suitable GPU kernels.
9. Is it useful for prefill? Yes, prefill uses full prompt attention.
10. FlashAttention vs KV cache? Training/prefill efficiency vs decode reuse.

17. Practice Tasks

Benchmark PyTorch SDPA. Compare memory with naive attention. Test causal vs non-causal masks. Increase sequence length until OOM. Explain online softmax.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Attention Benchmark	Compares attention kernels	PyTorch	Synthetic	Systems depth
Long Context Fine-Tune	Uses efficient attention	HF	Long documents	Practical LLM skill
Online Softmax Demo	Implements block softmax	NumPy	Synthetic	Math clarity

19. Quick Revision

Key idea: exact attention with lower memory IO. Formula: same softmax attention. Use for efficient long sequence training/prefill. Trap: saying it reduces quadratic compute.

20. Final Cheat Sheet

Definition: IO-aware tiled attention. Input/output: Q,K,V to attention output. Hyperparameters: block size, dtype, mask. Pros: faster/lower memory. Cons: still dense quadratic.

RAG

1. Overview

Retrieval-Augmented Generation (RAG) improves LLM answers by retrieving relevant external documents and adding them to the prompt. It is useful for private, current, or large knowledge that should not be memorized in model weights.

2. Intuition

Before answering, the model opens the right notes. The generator still writes the answer, but retrieval provides grounded context.

3. Prerequisites

Embeddings, vector search, chunking, prompts, LLM generation, evaluation, basic IR metrics.

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
Chunking	Split documents	Retrieval unit quality	500-token chunks	Chunk size tradeoff

Concept	Meaning	Why it matters	Example	Interview angle
Embeddings	Vector representations	Semantic search	dense vector	Similarity metrics
Retriever	Finds relevant chunks	Grounds answer	top-k search	Recall matters
Generator	LLM answerer	Uses context	final response	Hallucination control
Reranker	Reorders candidates	Improves precision	cross-encoder	Latency tradeoff

5. Algorithm / Working Process

Ingest documents, clean text, chunk, embed chunks, store vectors. At query time, embed query, retrieve top-k chunks, optionally rerank, build prompt with context, generate answer, cite sources, evaluate.

6. Mathematical Foundation

Cosine similarity:

$$\text{sim}(q, d) = \frac{q \cdot d}{||q|| ||d||}$$

RAG answer:

$$y = LLM(x, \text{retrieve}(x, D))$$

Retrieval metrics: recall@k, precision@k, MRR, nDCG. Generation metrics: faithfulness, answer correctness, citation accuracy.

7. Practical Implementation

```
from sentence_transformers import SentenceTransformer
import numpy as np

docs = [
    "LoRA trains low-rank adapter matrices.",
    "KV cache stores attention keys and values.",
    "RAG retrieves documents before generation.",
]

model = SentenceTransformer("all-MiniLM-L6-v2")
doc_vecs = model.encode(docs, normalize_embeddings=True)

query = "How does retrieval augmented generation work?"
q_vec = model.encode([query], normalize_embeddings=True)[0]
scores = doc_vecs @ q_vec
best = np.argsort(scores)[::-1][:2]

for i in best:
    print(float(scores[i]), docs[i])
```

8. Code Explanation

Documents and query are embedded into normalized vectors. Dot product equals cosine similarity. The highest-scoring chunks would be inserted into an LLM prompt.

9. Training / Evaluation

RAG often needs no LLM training. Evaluate retrieval separately from generation. Use gold answer/source pairs, recall@k, faithfulness checks, hallucination rate, latency, and user feedback.

10. Complexity and Cost

Indexing cost depends on document count and embedding model. Query cost includes embedding, vector search, reranking, and LLM tokens. Context length drives LLM cost.

11. Common Use Cases

Enterprise search, customer support, legal QA, medical literature assistants, codebase QA, policy assistants, knowledge-base chat.

12. Common Mistakes

Huge chunks, no metadata filters, poor PDF parsing, no reranking, stuffing too many chunks, evaluating only final answers, ignoring source citations, using RAG for facts already in prompt.

13. Edge Cases / Limitations

Bad retrieval causes bad answers. Conflicting documents confuse the model. Tables, images, code, and scanned PDFs need special parsing. Private data requires access control.

14. Variations

Naive RAG, hybrid search, query rewriting, multi-hop RAG, graph RAG, agentic RAG, corrective RAG, self-RAG.

15. Related Topics

RAG vs fine-tuning: RAG injects knowledge; fine-tuning changes behavior. Vector DBs store embeddings. Rerankers improve retrieval precision.

16. Interview Questions

1. What is RAG? Retrieval plus generation.
2. Why use RAG? Grounding and private/current knowledge.
3. What is chunking? Splitting docs into retrievable units.
4. What is top-k? Number of retrieved chunks.
5. What metric for retriever? Recall@k.
6. What causes hallucination in RAG? Missing or ignored evidence.
7. RAG or fine-tuning for new documents? Usually RAG.
8. Why rerank? Better relevance ordering.
9. What is hybrid search? Dense plus keyword retrieval.

10. How secure RAG? Enforce document-level permissions before generation.

17. Practice Tasks

Build a local RAG over markdown files. Test chunk sizes. Add metadata filters. Add source citations. Create failure cases where retrieval misses the answer.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Placement Notes QA	Answers from study notes	FAISS, HF	Your notes	Directly useful
PDF Policy Bot	Cites policy clauses	LangChain/LlamaIndex	Public PDFs	Enterprise RAG
Codebase Assistant	Answers repo questions	embeddings, FastAPI	Git repo	AI engineering

19. Quick Revision

Key idea: retrieve context before generating. Formula: cosine similarity. Use for external knowledge. Trap: poor chunking and no eval.

20. Final Cheat Sheet

Definition: retrieval-grounded LLM answering. Input/output: query to cited answer. Steps: chunk, embed, index, retrieve, generate. Hyperparameters: chunk size, top-k, reranker. Pros: current/private knowledge. Cons: retrieval failures.

Vector Databases

1. Overview

Vector databases store embeddings and support similarity search. They are core infrastructure for RAG, semantic search, recommendations, duplicate detection, and memory systems.

2. Intuition

Convert items into points in a high-dimensional space. Similar meanings land near each other. The database quickly finds nearest neighbors.

3. Prerequisites

Embeddings, cosine/dot/L2 similarity, indexing, metadata filtering, ANN search.

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
Vector	Numeric representation	Search unit	384-d embedding	Dense retrieval
ANN index	Approximate nearest neighbor	Fast search	HNSW, IVF	Recall/latency tradeoff
Metadata	Structured fields	Filtering/security	user_id, date	Access control
Upsert	Insert/update vector	Maintains index	document chunk	Data pipeline

5. Algorithm / Working Process

Embed documents, store vector plus text and metadata, build ANN index. At query time, embed query, apply metadata filters, search nearest vectors, return matching payloads.

6. Mathematical Foundation

Common distances:

$$\cos(q, d) = \frac{q \cdot d}{||q|| ||d||}$$

$$L2(q, d) = \sqrt{\sum_i (q_i - d_i)^2}$$

ANN trades exactness for speed: target high recall@k with low latency.

7. Practical Implementation

```
import faiss
import numpy as np

vectors = np.random.randn(1000, 384).astype("float32")
faiss.normalize_L2(vectors)

index = faiss.IndexFlatIP(384) # exact cosine if vectors are normalized
index.add(vectors)

query = np.random.randn(1, 384).astype("float32")
faiss.normalize_L2(query)
scores, ids = index.search(query, k=5)
print(ids[0], scores[0])
```

8. Code Explanation

Vectors are normalized, so inner product behaves like cosine similarity. FAISS index stores vectors and returns nearest IDs and scores.

9. Training / Evaluation

Vector DBs are not trained like models, but indexes are tuned. Evaluate recall@k, latency, throughput, update speed, storage, filter correctness, and security.

10. Complexity and Cost

Exact search is $O(Nd)$. ANN indexes reduce search time using extra memory and approximate results. Cost depends on vector count, dimension, replicas, and metadata indexes.

11. Common Use Cases

RAG, semantic search, recommendations, image search, deduplication, clustering, personalization memory.

12. Common Mistakes

Wrong similarity metric, forgetting normalization, no metadata filters, re-embedding with a different model without rebuilding index, ignoring deletes, using vector search when keyword search is better.

13. Edge Cases / Limitations

Embeddings can miss exact terms, numbers, and rare identifiers. ANN may miss nearest neighbors. High-dimensional indexes need memory. Access control must be enforced outside pure similarity.

14. Variations

FAISS, Milvus, Weaviate, Pinecone, Qdrant, Chroma, pgvector, Elasticsearch dense vectors. HNSW for graph ANN, IVF for clustered ANN, PQ for compressed vectors.

15. Related Topics

Vector DBs power RAG. Embedding models create vectors. Hybrid search combines vector and BM25.

16. Interview Questions

1. What is a vector DB? A database optimized for embedding similarity search.
2. What is ANN? Approximate nearest neighbor search.
3. Why approximate? Faster search at scale.
4. Cosine vs dot product? Same if vectors are normalized.
5. Why store metadata? Filtering, citations, security.
6. What is HNSW? Graph-based ANN index.
7. What is recall@k? Whether relevant item appears in top k.
8. What happens if embedding model changes? Rebuild embeddings/index.
9. Vector vs keyword search? Semantic vs lexical matching.
10. Why use pgvector? Simple vector search inside Postgres.

17. Practice Tasks

Build FAISS index. Add metadata mapping. Compare exact vs HNSW. Test cosine vs L2. Evaluate recall on labeled queries.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Semantic Notes Search	Searches study notes	FAISS	Markdown notes	RAG foundation
Image Similarity Search	Finds similar images	CLIP, FAISS	CIFAR/images	Multimodal skill
Hybrid Search API	Dense plus keyword	FastAPI, pgvector	Docs	Production relevance

19. Quick Revision

Key idea: nearest-neighbor search over embeddings. Formula: cosine similarity. Use for semantic retrieval.
Trap: metric/model mismatch.

20. Final Cheat Sheet

Definition: embedding similarity store. Input/output: vector query to nearest items. Steps: embed, index, search, filter. Hyperparameters: metric, k, index type. Pros: semantic search. Cons: approximate and embedding-dependent.

Embedding Models

1. Overview

Embedding models convert text, images, audio, or other data into dense vectors where semantic similarity is reflected by geometric closeness. They power RAG, search, clustering, classification features, recommendations, and deduplication.

2. Intuition

An embedding is a coordinate for meaning. "car" and "vehicle" should be closer than "car" and "banana" in vector space.

3. Prerequisites

Neural networks, Transformers, contrastive learning, cosine similarity, tokenization.

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
Dense vector	Fixed-size representation	Enables search	768 dims	Why fixed size

Concept	Meaning	Why it matters	Example	Interview angle
Contrastive learning	Pull positives together	Trains semantic space	query-doc pairs	InfoNCE
Pooling	Convert token states to vector	Sentence representation	mean pooling	CLS vs mean
Normalization	Unit-length vectors	Stable cosine search	L2 norm	Dot vs cosine

5. Algorithm / Working Process

Tokenize input, pass through encoder, pool token embeddings, optionally project and normalize, then compare vectors using cosine/dot/L2.

6. Mathematical Foundation

Cosine similarity:

$$\text{sim}(a, b) = \frac{a \cdot b}{||a|| ||b||}$$

Contrastive loss for one positive among negatives:

$$L = -\log \frac{\exp(\text{sim}(q, d^+)/\tau)}{\sum_j \exp(\text{sim}(q, d_j)/\tau)}$$

7. Practical Implementation

```
from sentence_transformers import SentenceTransformer, util

model = SentenceTransformer("all-MiniLM-L6-v2")
sentences = ["LoRA adapts LLMs.", "Adapters fine-tune models cheaply.", "I like pizza."]
emb = model.encode(sentences, normalize_embeddings=True)

scores = util.cos_sim(emb[0], emb)
print(scores)
```

8. Code Explanation

The model maps each sentence to a vector. Normalization makes cosine comparison simple. Similar technical sentences get higher similarity.

9. Training / Evaluation

Train with pairs/triplets/query-document data. Evaluate retrieval recall@k, MTEB-style benchmarks, clustering purity, classification performance, and domain-specific search relevance.

10. Complexity and Cost

Embedding cost is roughly proportional to input tokens and model size. Storage is $N \times d \times \text{bytes}$. Larger dimensions can improve quality but increase index memory.

11. Common Use Cases

Semantic search, RAG, recommendations, clustering, duplicate detection, topic modeling, image-text retrieval.

12. Common Mistakes

Using generic embeddings for specialized domains, forgetting normalization, comparing embeddings from different models, ignoring chunk boundaries, using embeddings for exact numeric matching.

13. Edge Cases / Limitations

Embeddings may fail on rare terms, IDs, exact negation, numbers, and domain jargon. They can encode bias. Long inputs may be truncated.

14. Variations

Bi-encoders for fast retrieval. Cross-encoders for reranking. Multilingual embeddings. Multimodal embeddings like CLIP. Sparse embeddings like SPLADE.

15. Related Topics

Embedding models feed vector DBs and RAG. Cross-encoders trade speed for accuracy. Token embeddings inside LLMs are different from sentence embeddings.

16. Interview Questions

1. What is an embedding? Dense numeric representation.
2. Why use embeddings? Semantic comparison.
3. What is cosine similarity? Angle-based similarity.
4. Bi-encoder vs cross-encoder? Fast independent encoding vs joint scoring.
5. What is contrastive learning? Pull positives close, push negatives away.
6. Why normalize embeddings? Stable similarity scale.
7. What is dimension? Length of vector.
8. Can embeddings handle exact numbers well? Often poorly.
9. How evaluate retrieval embeddings? Recall@k, MRR, nDCG.
10. What is CLIP? Multimodal image-text embedding model.

17. Practice Tasks

Compute sentence similarities. Build a semantic search demo. Fine-tune on pairs. Test domain jargon. Compare embedding models.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Resume Matcher	Matches resumes to jobs	SBERT, FAISS	Job posts	Placement relevance
Duplicate Detector	Finds similar tickets	embeddings	Issue data	Practical ML
Multilingual Search	Searches across languages	multilingual model	Wikipedia	NLP depth

19. Quick Revision

Key idea: map meaning to vectors. Formula: cosine similarity, contrastive loss. Use for retrieval/search. Trap: assuming semantic search is exact search.

20. Final Cheat Sheet

Definition: model producing dense representations. Input/output: text/image to vector. Hyperparameters: dimension, pooling, model, max length. Pros: semantic matching. Cons: weak exact reasoning.

Function Calling / Tool Use

1. Overview

Function calling or tool use lets an LLM request structured external actions, such as searching a database, calling an API, running code, sending email, or retrieving live data. It turns a language model from a pure text generator into a controller over tools.

2. Intuition

The LLM decides when it needs a calculator, database, or API instead of guessing. It outputs structured arguments, the system executes the tool, and the model uses the result.

3. Prerequisites

JSON/schema design, APIs, prompts, LLM inference, validation, security boundaries.

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
Tool schema	Function name and arguments	Constrains output	<code>get_weather(city)</code>	Why schemas
Tool router	Chooses tool	Controls workflow	search vs calculator	Tool selection

Concept	Meaning	Why it matters	Example	Interview angle
Execution layer	Runs function	Separates model from action	API call	Security
Observation	Tool result returned to model	Enables final answer	JSON output	Multi-step loops

5. Algorithm / Working Process

Define available tools and schemas. User asks a task. Model decides whether to call a tool. System validates arguments, executes tool, returns result. Model writes final answer or calls another tool.

6. Mathematical Foundation

Tool use is usually trained as conditional generation:

$$P(y|x, S)$$

where S is tool schema context. The model learns to emit either text or structured tool-call tokens. Evaluation uses exact argument match, tool success rate, and task completion.

7. Practical Implementation

```
import json

def add(a: float, b: float) -> float:
    return a + b

tool_call = '{"name": "add", "arguments": {"a": 2, "b": 3}}'
call = json.loads(tool_call)

if call["name"] == "add":
    args = call["arguments"]
    result = add(args["a"], args["b"])
    print({"tool_result": result})
```

8. Code Explanation

The model would produce the JSON. The application parses and validates it, executes the matching function, and returns the result for final response generation.

9. Training / Evaluation

Evaluate tool-call precision, argument correctness, invalid-call rate, task success, latency, and safety. Use integration tests with mocked tools.

10. Complexity and Cost

Tool use adds orchestration latency and failure modes. It can reduce LLM token cost by delegating exact work to APIs. External tools may have rate limits and security costs.

11. Common Use Cases

Database querying, booking, code execution, calendars, email, search, calculators, CRM actions, RAG retrieval.

12. Common Mistakes

Letting model execute unsafe actions, weak argument validation, too many overlapping tools, no permission checks, trusting tool output blindly, hiding tool errors.

13. Edge Cases / Limitations

Ambiguous user requests, missing required arguments, tool outages, prompt injection in tool results, irreversible actions, stale schemas.

14. Variations

Single-call tools, multi-step agents, ReAct-style reasoning/action loops, structured outputs, constrained decoding, MCP-style tool connectors.

15. Related Topics

Tool use powers agentic workflows. RAG can be implemented as a retrieval tool. Structured output is related but may not execute actions.

16. Interview Questions

1. What is function calling? Structured LLM request to execute a function.
2. Why use schemas? Valid arguments and predictable parsing.
3. Who executes the tool? The application, not the model.
4. Main risk? Unsafe or invalid actions.
5. How validate? JSON schema, types, permissions.
6. Tool use vs RAG? RAG retrieves context; tool use can do any external action.
7. What is observation? Tool result returned to model.
8. How test? Mock tools and assert calls.
9. What is prompt injection risk? Tool data may instruct the model maliciously.
10. Why not let LLM guess live data? Tools provide exact/current data.

17. Practice Tasks

Build a calculator tool loop. Add schema validation. Mock a weather API. Add permission confirmation for destructive actions. Test invalid arguments.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
SQL Analyst Agent	Calls SQL tool	FastAPI, SQLite	Sample DB	Enterprise AI

Project	What it does	Tech stack	Dataset	Resume value
Calendar Assistant	Schedules events	API tools	Synthetic	Workflow automation
Safe Tool Sandbox	Validates tool calls	Python	Test prompts	Security awareness

19. Quick Revision

Key idea: LLM emits structured calls to external functions. Use for exact actions/data. Trap: no validation or permissions.

20. Final Cheat Sheet

Definition: model-controlled external function execution. Input/output: user request to tool call/result/final answer. Hyperparameters: tool set, schema, max steps. Pros: exact actions. Cons: orchestration and safety risk.

Mixture of Experts

1. Overview

Mixture of Experts (MoE) models contain multiple expert subnetworks and route each token to a small subset of them. They increase model capacity without activating all parameters for every token.

2. Intuition

Instead of one giant team working on every token, a router sends each token to the most relevant specialists.

3. Prerequisites

Transformers, feed-forward networks, routing, softmax, load balancing, distributed training.

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
Expert	Usually an MLP block	Adds capacity	8 experts	Active vs total params
Router/gate	Chooses experts	Controls specialization	top-2 routing	Load balance
Sparse activation	Use few experts per token	Efficient compute	top-k	Why cheaper
Load balancing loss	Prevents expert collapse	Uses all experts	auxiliary loss	Training stability

5. Algorithm / Working Process

For each token, router computes expert scores. Select top-k experts. Send token representation to those experts. Combine expert outputs using routing weights. Add load-balancing loss during training.

6. Mathematical Foundation

Router:

$$p_i = \text{softmax}(W_r h)_i$$

Top-k experts E_k are selected:

$$y = \sum_{i \in \text{topk}(p)} p_i E_i(h)$$

Training loss:

$$L = L_{LM} + \lambda L_{balance}$$

7. Practical Implementation

```
import torch
import torch.nn as nn

class TinyMoE(nn.Module):
    def __init__(self, dim, experts=4):
        super().__init__()
        self.router = nn.Linear(dim, experts)
        self.experts = nn.ModuleList([nn.Sequential(nn.Linear(dim, dim),
nn.ReLU()) for _ in range(experts)])

    def forward(self, x):
        probs = torch.softmax(self.router(x), dim=-1)
        expert_id = probs.argmax(dim=-1)
        out = torch.zeros_like(x)
        for i, expert in enumerate(self.experts):
            mask = expert_id == i
            if mask.any():
                out[mask] = expert(x[mask]) * probs[mask, i].unsqueeze(-1)
        return out
```

8. Code Explanation

The router scores experts for each token. This toy version sends each token to one expert. Real MoE uses optimized dispatch, top-k routing, and load-balancing losses.

9. Training / Evaluation

Evaluate perplexity, downstream accuracy, expert utilization, routing entropy, throughput, and communication overhead. Distributed training must monitor imbalance.

10. Complexity and Cost

MoE has many total parameters but fewer active parameters per token. Training and serving need more memory for all experts and communication across devices.

11. Common Use Cases

Large-scale LLMs, multilingual models, multi-domain models, compute-efficient scaling.

12. Common Mistakes

Confusing total and active parameters, ignoring routing imbalance, assuming experts are human-interpretable, underestimating serving complexity, no capacity management.

13. Edge Cases / Limitations

Expert collapse, token dropping when capacity is full, communication bottlenecks, unstable training, hard deployment.

14. Variations

Top-1 routing, top-2 routing, Switch Transformer, GShard, Mixtral-style sparse MoE, dense-MoE hybrids.

15. Related Topics

MoE vs dense Transformer: more capacity with sparse compute. Related to conditional computation and routing.

16. Interview Questions

1. What is MoE? Multiple experts with sparse routing.
2. Why use MoE? More capacity without proportional compute.
3. What is active parameter count? Parameters used per token.
4. What does router do? Selects experts.
5. What is expert collapse? Router sends most tokens to few experts.
6. Why load balancing loss? Spread traffic.
7. Is MoE easier to serve? No, often harder.
8. Are experts interpretable? Not necessarily.
9. Top-1 vs top-2? One expert vs weighted two experts.
10. Main bottleneck? Routing and distributed communication.

17. Practice Tasks

Implement tiny MoE. Track expert usage. Add top-2 routing. Add load-balance penalty. Compare dense MLP vs MoE.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Tiny MoE Classifier	Routes samples to experts	PyTorch	AG News	Architecture skill
Expert Utilization Dashboard	Visualizes routing	PyTorch, Streamlit	Synthetic	Debugging skill
Dense vs MoE Study	Compares compute/quality	PyTorch	small LM data	Research angle

19. Quick Revision

Key idea: route tokens to a few experts. Formula: weighted expert sum. Use for scalable capacity. Trap: ignoring serving complexity.

20. Final Cheat Sheet

Definition: sparse expert architecture. Input/output: token states to routed expert outputs. Hyperparameters: experts, top-k, capacity, balance loss. Pros: high capacity. Cons: complex routing/serving.

Speculative Decoding

1. Overview

Speculative decoding accelerates LLM inference by using a small draft model to propose tokens and a larger target model to verify them. It preserves the target model distribution when implemented correctly.

2. Intuition

A fast assistant drafts several words. The expert checks them in one pass and accepts as many as possible. If the draft is good, generation speeds up.

3. Prerequisites

Autoregressive decoding, sampling distributions, acceptance-rejection, batching, KV cache.

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
Draft model	Smaller/faster model	Proposes tokens cheaply	1B model	Speed depends on match
Target model	Original model	Verifies quality	7B model	Distribution preservation
Acceptance	Decide which tokens to keep	Correctness	accept 3 of 5	Why exact

Concept	Meaning	Why it matters	Example	Interview angle
Lookahead	Number of draft tokens	Speed/accept tradeoff	gamma=4	Tuning

5. Algorithm / Working Process

Draft model generates several candidate tokens. Target model evaluates them in parallel. Accepted tokens are appended. If a token is rejected, sample a correction from adjusted target distribution. Repeat until done.

6. Mathematical Foundation

For draft distribution q and target p , accept token x with:

$$a = \min(1, p(x)/q(x))$$

If rejected, sample from a corrected residual distribution. This preserves target-model sampling.

7. Practical Implementation

```
# Toy greedy speculative idea, not exact sampling.
def speculative_step(prefix, draft_generate, target_accepts, gamma=4):
    draft_tokens = draft_generate(prefix, max_new_tokens=gamma)
    accepted = []
    for token in draft_tokens:
        if target_accepts(prefix + accepted, token):
            accepted.append(token)
        else:
            break
    return accepted
```

8. Code Explanation

The draft proposes multiple tokens. The target checker accepts a prefix of them. Production implementations verify probabilities in parallel and handle rejection mathematically.

9. Training / Evaluation

No training required if using existing draft and target models. Evaluate speedup, acceptance rate, output equality/distribution, latency, and memory. Draft must be much cheaper and reasonably aligned.

10. Complexity and Cost

Speedup comes from verifying multiple tokens in one target forward pass. Gains shrink if draft quality is poor or target verification dominates.

11. Common Use Cases

Chat serving, code generation, low-latency assistants, high-throughput APIs.

12. Common Mistakes

Using a weak draft model, claiming quality changes are expected, ignoring sampling correctness, choosing too large lookahead, measuring without realistic batching.

13. Edge Cases / Limitations

Low acceptance on creative/high-temperature tasks. Extra draft model memory. Complex integration with batching and cache management.

14. Variations

N-gram speculation, Medusa-style multiple heads, self-speculative decoding, prompt lookup decoding, tree-based speculation.

15. Related Topics

KV cache speeds sequential decode; speculative decoding reduces target decode steps. Distillation can produce a better draft model.

16. Interview Questions

1. What is speculative decoding? Draft then verify tokens.
2. Does it change model quality? Correct versions preserve target distribution.
3. Why faster? Target verifies multiple tokens per pass.
4. What is acceptance rate? Fraction of draft tokens kept.
5. What makes a good draft? Fast and similar to target.
6. What is gamma? Draft lookahead length.
7. Main limitation? Poor acceptance or extra memory.
8. Is it training? Usually inference only.
9. Why exact sampling matters? Output distribution should match target.
10. Relation to KV cache? Both optimize decoding.

17. Practice Tasks

Simulate draft/target acceptance. Measure speedup vs acceptance. Implement greedy toy version. Compare draft sizes. Plot gamma tradeoff.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Spec Decode Simulator	Shows acceptance/speed	Python	Synthetic	Inference insight
Draft Model Benchmark	Compares drafts	HF	Prompt set	Serving skill
Medusa Explainer	Visualizes multi-token heads	PyTorch	Toy LM	Research relevance

19. Quick Revision

Key idea: small model proposes, big model verifies. Formula: accept with $\min(1, p/q)$. Use for faster decoding. Trap: assuming approximate output is okay.

20. Final Cheat Sheet

Definition: verified draft-token decoding. Input/output: prefix to multiple accepted tokens. Hyperparameters: draft model, gamma, temperature. Pros: lower latency. Cons: draft memory and acceptance dependence.

Long-Context Methods

1. Overview

Long-context methods allow Transformers and LLMs to process more tokens than standard training lengths. They are important for document QA, codebase analysis, legal review, long conversations, and video/audio understanding.

2. Intuition

A normal model has a small desk. Long-context methods either build a bigger desk, summarize older papers, retrieve only relevant pages, or use smarter attention so the desk is not overwhelmed.

3. Prerequisites

Transformers, attention complexity, positional encodings, RAG, memory usage, evaluation.

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
Context window	Max tokens model can attend to	Hard inference limit	32k tokens	Token budgeting
Positional encoding	Represents token positions	Needed beyond train length	RoPE	Extrapolation
Sparse/sliding attention	Restricts attention pattern	Reduces cost	local window	Tradeoff
Retrieval	Select relevant context	Avoids stuffing	RAG	Often better

5. Algorithm / Working Process

Options include training with longer sequences, extending positional embeddings, using efficient attention kernels, sliding-window attention, recurrence/memory, summarization, or retrieval-based context selection.

6. Mathematical Foundation

Dense attention cost:

$$O(N^2d)$$

Sliding-window attention with window w :

$$O(Nwd)$$

RoPE modifies query/key representations using position-dependent rotations, enabling relative position behavior.

7. Practical Implementation

```
def sliding_chunks(tokens, window=512, stride=256):
    chunks = []
    for start in range(0, len(tokens), stride):
        chunk = tokens[start:start + window]
        if chunk:
            chunks.append(chunk)
        if start + window >= len(tokens):
            break
    return chunks

tokens = list(range(1200))
print([len(c) for c in sliding_chunks(tokens)])
```

8. Code Explanation

The function creates overlapping windows. This is a simple baseline for long documents when the model cannot process everything at once.

9. Training / Evaluation

Evaluate needle-in-haystack retrieval, long-document QA, summarization consistency, position bias, lost-in-the-middle behavior, latency, memory, and answer citation.

10. Complexity and Cost

Long context is expensive because attention and KV cache grow with sequence length. Even with FlashAttention, dense attention remains quadratic in compute.

11. Common Use Cases

Long PDF QA, codebase chat, legal contracts, research paper analysis, meeting transcripts, long user memory.

12. Common Mistakes

Stuffing irrelevant context, ignoring lost-in-the-middle, using long context instead of retrieval, no position-based eval, exceeding KV memory.

13. Edge Cases / Limitations

Models may miss facts in the middle, struggle with multi-hop reasoning, or become expensive. Longer context does not guarantee better use of context.

14. Variations

RoPE scaling, ALiBi, sliding-window attention, recurrent memory, Transformer-XL, Longformer/BigBird, RAG, summarization memory.

15. Related Topics

RAG often beats brute-force long context. FlashAttention helps long prefill. KV cache becomes a bottleneck.

16. Interview Questions

1. Why is long context hard? Attention and memory scale badly.
2. What is lost-in-the-middle? Model ignores middle context.
3. Dense attention complexity? $O(N^2d)$.
4. Sliding attention complexity? $O(Nwd)$.
5. RAG vs long context? Retrieval selects relevant context; long context includes more raw text.
6. What is RoPE scaling? Extending positional behavior.
7. Does long context guarantee factuality? No.
8. Main serving cost? KV cache memory.
9. How evaluate? Long QA and needle tests.
10. When use summarization? For persistent conversation memory.

17. Practice Tasks

Chunk a long document. Test retrieval vs stuffing. Build a lost-in-middle prompt. Measure token cost. Summarize conversation memory.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Long PDF QA	Answers long docs	RAG, long LLM	arXiv PDFs	Applied LLM
Needle Benchmark	Tests retrieval by position	Python	Synthetic	Evaluation skill
Codebase Context Tool	Selects relevant files	embeddings	Git repos	AI engineering

19. Quick Revision

Key idea: handle more tokens using bigger windows, efficient attention, memory, or retrieval. Formula: dense attention $O(N^2)$. Trap: more context is not always better.

20. Final Cheat Sheet

Definition: methods for extended sequence processing. Input/output: long input to answer/summary.

Hyperparameters: context length, chunk size, stride, window. Pros: handles large docs. Cons: expensive and position-sensitive.

Multimodal LLMs

1. Overview

Multimodal LLMs process text plus other modalities such as images, audio, video, documents, or sensor data. They are used for visual question answering, OCR reasoning, chart understanding, image captioning, voice assistants, and robotics.

2. Intuition

The model gets eyes or ears through encoders that convert non-text inputs into representations the language model can reason over.

3. Prerequisites

Transformers, CNN/ViT encoders, embeddings, cross-attention, tokenization, contrastive learning.

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
Modality encoder	Converts image/audio to features	Bridges input type	ViT	Why not raw pixels
Projection layer	Maps features to LLM space	Connects encoder to LLM	MLP adapter	Alignment
Cross-attention	Text attends to visual tokens	Multimodal fusion	Flamingo-style	Fusion methods
Instruction tuning	Teaches multimodal responses	Usability	VQA data	Data quality

5. Algorithm / Working Process

Encode image/audio/video into feature tokens. Project features into LLM embedding dimension. Combine with text tokens using prefix tokens, cross-attention, or unified tokenization. Generate text answer autoregressively.

6. Mathematical Foundation

Image encoder:

$$Z_v = Enc_v(image)$$

Projection:

$$H_v = W_p Z_v$$

Generation:

$$P(y|x_{text}, image) = \prod_t P(y_t|y_{<t}, x_{text}, H_v)$$

Training often combines captioning loss, VQA loss, contrastive loss, and instruction tuning.

7. Practical Implementation

```
from PIL import Image
from transformers import BlipProcessor, BlipForConditionalGeneration

processor = BlipProcessor.from_pretrained("Salesforce/blip-image-captioning-base")
model = BlipForConditionalGeneration.from_pretrained("Salesforce/blip-image-captioning-base")

image = Image.new("RGB", (224, 224), color="white")
inputs = processor(image, return_tensors="pt")
out = model.generate(**inputs, max_new_tokens=20)
print(processor.decode(out[0], skip_special_tokens=True))
```

8. Code Explanation

The processor prepares image tensors. The model encodes the image and generates a caption. Larger multimodal chat models follow the same broad encode-project-generate pattern.

9. Training / Evaluation

Datasets include image-caption pairs, VQA, OCR documents, charts, and instruction data. Metrics include VQA accuracy, CIDEr/BLEU for captions, OCR accuracy, hallucination rate, and human evaluation.

10. Complexity and Cost

Vision/video tokens increase context and compute. High-resolution images need tiling or patching. Video is expensive because frames multiply tokens.

11. Common Use Cases

Visual QA, document understanding, chart analysis, accessibility captioning, medical image assistance, UI automation, robotics.

12. Common Mistakes

Assuming visual grounding is perfect, ignoring OCR errors, evaluating only captions, using low-resolution images for text-heavy tasks, missing safety issues for medical/legal images.

13. Edge Cases / Limitations

Small text, crowded charts, spatial reasoning, counting, medical diagnosis, adversarial images, and long videos remain hard.

14. Variations

CLIP-style dual encoders, BLIP-style captioners, LLaVA-style visual instruction tuning, Flamingo-style cross-attention, unified multimodal token models.

15. Related Topics

Embedding models connect image/text spaces. RAG can retrieve multimodal documents. Function calling can let models use OCR or image tools.

16. Interview Questions

1. What is a multimodal LLM? LLM that handles text plus other modalities.
2. Why use an image encoder? To convert pixels to features.
3. What is projection? Mapping visual features to LLM dimension.
4. CLIP vs VQA model? Retrieval alignment vs question answering.
5. Why is OCR important? Many images contain text.
6. Main limitation? Hallucinated visual details.
7. How handle video? Sample frames and encode temporal info.
8. What metrics? VQA accuracy, caption metrics, human eval.
9. What is visual instruction tuning? Training on image-instruction-answer data.
10. Why high-res tiling? Preserve small details.

17. Practice Tasks

Run image captioning. Build VQA demo. Compare low/high-resolution inputs. Test chart questions. Add OCR preprocessing.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Chart QA Bot	Answers chart questions	BLIP/LLaVA, OCR	ChartQA	Strong portfolio
Receipt Extractor	Reads receipts	OCR, LLM	SROIE	Applied AI
Image Search	Text-to-image retrieval	CLIP, FAISS	COCO	Multimodal retrieval

19. Quick Revision

Key idea: encode non-text into tokens/features the LLM can use. Formula: $P(y|text, image)$. Use for visual/audio/document tasks. Trap: trusting hallucinated visual claims.

20. Final Cheat Sheet

Definition: LLM with non-text inputs. Input/output: image/audio/text to text/action. Hyperparameters: resolution, visual tokens, fusion method. Pros: broader tasks. Cons: high compute and grounding errors.

Agentic Workflows

1. Overview

Agentic workflows use LLMs to plan, call tools, inspect results, and iterate toward a goal. They are used for coding agents, research assistants, data-analysis agents, customer operations, and workflow automation.

2. Intuition

A normal LLM answers once. An agent can make a plan, use tools, check what happened, and continue.

3. Prerequisites

Prompting, function calling, APIs, state machines, evaluation, security, RAG.

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
Planner	Breaks task into steps	Handles complex goals	research plan	Planning failure
Tool use	External actions	Extends ability	search, code	Validation
Memory/state	Stores progress	Multi-step continuity	scratchpad	What to persist
Reflection/eval	Checks outputs	Reduces errors	test after edit	Self-correction limits

5. Algorithm / Working Process

Receive goal, gather context, plan minimal steps, call tools, observe outputs, update state, verify result, stop when done or ask for human input when blocked.

6. Mathematical Foundation

Agent loop can be modeled as a policy over actions:

$$a_t \sim \pi(a|s_t, g)$$

State update:

$$s_{t+1} = Env(s_t, a_t)$$

Objective is task success under cost and safety constraints.

7. Practical Implementation

```
def run_agent(goal, tools, max_steps=5):
    state = {"goal": goal, "notes": []}
    for _ in range(max_steps):
        if "weather" in goal.lower():
            result = tools["weather"]("Delhi")
```

```
        return f"Weather result: {result}"
    state["notes"].append("No matching tool; answer directly.")
    return "I can answer this without a tool."
return "Stopped after max steps."
```

8. Code Explanation

This tiny loop shows the structure: keep state, choose an action, call a tool, return or continue. Real agents use LLM calls for action selection and stricter validation.

9. Training / Evaluation

Evaluate task success rate, tool-call correctness, cost, latency, number of steps, rollback safety, and human intervention rate. Use realistic end-to-end tests.

10. Complexity and Cost

Agents can be expensive because each step may call an LLM and tools. Long loops amplify latency and error. Strong stopping conditions save cost.

11. Common Use Cases

Coding agents, web research, data analysis, report generation, IT automation, support workflows, personal assistants.

12. Common Mistakes

No stop condition, too many tools, no permissions, no verification, treating reasoning text as truth, no audit logs, allowing prompt injection through tool output.

13. Edge Cases / Limitations

Long-horizon tasks drift. Tools fail. Agents can loop, call wrong tools, or take unsafe actions. Human approval is needed for irreversible operations.

14. Variations

ReAct, plan-and-execute, reflexion, multi-agent workflows, graph-based agents, state-machine agents, human-in-the-loop agents.

15. Related Topics

Function calling is the action layer. RAG is a retrieval tool. Post-training improves tool-use reliability. MLOps handles monitoring.

16. Interview Questions

1. What is an agentic workflow? LLM loop with planning/tools/state.
2. Agent vs chatbot? Agent acts and iterates.
3. Why use tools? Exact external operations.

4. Main risk? Unsafe actions and compounding errors.
5. How evaluate? End-to-end task success.
6. What is ReAct? Reason/action/observation pattern.
7. Why limit steps? Cost and loop control.
8. What needs human approval? Destructive or sensitive actions.
9. What is memory? Persisted task/user/context state.
10. How reduce hallucination? Grounding, tools, verification.

17. Practice Tasks

Build a tool-calling loop. Add max-step limit. Add approval for delete actions. Add logging. Evaluate on 20 tasks.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Data Analyst Agent	Loads CSV and answers	Python, pandas, LLM	Kaggle CSVs	AI engineer role
Coding Fix Agent	Runs tests and patches	Python, git	Toy repo	Agentic coding
Research Agent	Searches and summarizes	RAG, tools	Web/docs	Workflow automation

19. Quick Revision

Key idea: LLM chooses actions over multiple steps. Formula: policy over actions. Use for tool-heavy workflows. Trap: no verification or permissions.

20. Final Cheat Sheet

Definition: iterative LLM planning/tool loop. Input/output: goal to completed task. Hyperparameters: tools, max steps, memory, approvals. Pros: handles workflows. Cons: brittle and costly.

Post-Training Pipelines

1. Overview

Post-training pipelines transform a pretrained base LLM into a useful assistant. They usually include continued pretraining, supervised fine-tuning, preference optimization, safety tuning, tool-use training, evaluation, and deployment checks.

2. Intuition

Pretraining gives broad language ability. Post-training shapes that ability into a product: follows instructions, refuses unsafe requests, uses tools, writes in desired style, and passes evaluations.

3. Prerequisites

Pretraining, SFT, RLHF/DPO, dataset curation, evaluation, safety, PEFT, distributed training.

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
Continued pretraining	More next-token training on domain data	Adds domain fluency	code/legal corpus	Risk of forgetting
SFT	Train on instruction-response examples	Teaches format and tasks	chat data	First alignment step
Preference tuning	Optimize chosen over rejected	Improves quality	DPO/RLHF	Alignment
Safety tuning	Refusal and policy behavior	Reduces harmful outputs	red-team data	Over-refusal
Evaluation gate	Blocks regressions	Production readiness	benchmark suite	Release process

5. Algorithm / Working Process

Collect and clean data. Continue pretraining if domain language is missing. Run SFT for instruction behavior. Apply DPO/RLHF for preferences. Add safety and tool-use data. Evaluate across capability, safety, robustness, and latency. Deploy with monitoring.

6. Mathematical Foundation

SFT loss:

$$L_{SFT} = - \sum_t \log P_{\theta}(y_t | x, y_{<t})$$

Preference loss may use DPO:

$$L_{DPO} = - \log \sigma(\beta(\Delta \log \pi_{\theta} - \Delta \log \pi_{ref}))$$

Final model selection optimizes a multi-objective score:

$$Score = capability - \lambda_1 safety_risk - \lambda_2 latency - \lambda_3 cost$$

7. Practical Implementation

```
pipeline = [
    "deduplicate data",
    "filter low-quality samples",
    "supervised fine-tune",
    "preference tune with DPO",
```

```
    "run safety and capability evals",  
    "quantize or serve",  
    "monitor production feedback",  
]  
  
for step in pipeline:  
    print(f"Run: {step}")
```

8. Code Explanation

The code is a minimal checklist representation. Real pipelines implement each step with data jobs, training jobs, eval jobs, model registry, and deployment gates.

9. Training / Evaluation

Use held-out instruction data, preference sets, safety prompts, domain tests, adversarial prompts, regression tests, and human evaluation. Track contamination and data leakage.

10. Complexity and Cost

Post-training can be cheaper than pretraining but still expensive. Data quality and evaluation cost often dominate. DPO/PEFT are cheaper than full RLHF and full fine-tuning.

11. Common Use Cases

Building chat models, enterprise assistants, domain LLMs, coding models, support agents, tool-using assistants.

12. Common Mistakes

Skipping data deduplication, training on eval data, overfitting preference style, no safety regression, ignoring latency, mixing incompatible chat templates, poor versioning.

13. Edge Cases / Limitations

Post-training cannot fully fix a weak base model. Too much safety tuning can cause over-refusal. Domain tuning can reduce general ability.

14. Variations

SFT-only pipeline, SFT+DPO, SFT+RLHF, constitutional AI, tool-use post-training, domain-adaptive continued pretraining, PEFT-based pipeline.

15. Related Topics

RLHF and DPO are preference stages. LoRA/QLoRA/PEFT reduce training cost. Quantization improves deployment. RAG may replace some knowledge fine-tuning.

16. Interview Questions

1. What is post-training? Steps after pretraining to make a usable model.
2. What is SFT? Training on instruction-response examples.
3. Why preference tune? Improve outputs according to human choices.
4. Continued pretraining vs SFT? Domain language learning vs instruction behavior.
5. Why eval gates? Prevent regressions.
6. What is data contamination? Eval examples appear in training.
7. PEFT role? Cheap adaptation.
8. Safety tuning risk? Over-refusal.
9. RAG or post-train for new facts? Usually RAG.
10. Final deployment checks? Quality, safety, latency, cost.

17. Practice Tasks

Design a post-training plan for a domain assistant. Build a small SFT dataset. Add preference pairs. Create eval prompts. Track model versions.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Mini Chat Pipeline	SFT then DPO	TRL, PEFT	Alpaca + pairs	End-to-end LLM
Safety Eval Gate	Blocks bad checkpoints	Python	Red-team prompts	Responsible AI
Domain Assistant Pipeline	Tunes and evaluates	QLoRA, RAG	Public domain docs	Production story

19. Quick Revision

Key idea: shape pretrained LLM into a deployable assistant. Formulas: SFT loss and preference loss. Use for product-quality models. Trap: weak eval and contaminated data.

20. Final Cheat Sheet

Definition: training/eval stages after pretraining. Input/output: base model to assistant model. Steps: data, SFT, preference, safety, eval, deploy. Hyperparameters: LR, epochs, beta, data mix. Pros: aligned behavior. Cons: data/eval heavy.